

Hybrid ANN-HMM Architecture for Offline Handwriting Recognition

Natural Language Processing Project Documentation

Academic Year 2025-2026

Paraschiv Tudor

Mezei Robert

January 12, 2026

Abstract

This document presents a comprehensive analysis of our implementation of a Hybrid Artificial Neural Network and Hidden Markov Model (ANN-HMM) system for offline handwriting recognition. The project aimed to replicate classical approaches to handwriting recognition using Expectation-Maximization training on the IAM Handwriting Database. Throughout this documentation, we detail the theoretical foundations, implementation challenges, and experimental results of our system. Despite significant effort spanning multiple weeks of development and debugging, our implementation did not achieve competitive results compared to state-of-the-art methods. We document our struggles, partial successes, and the lessons learned from this challenging endeavor. The final system achieved a Character Error Rate (CER) of approximately 68-75%, which, while demonstrating some learning capability, falls significantly short of practical usability. This documentation serves as both a technical reference and an honest account of the difficulties inherent in implementing classical machine learning systems from scratch.

Contents

1	Problem Statement	4
1.1	Introduction to the NLP Task	4
1.2	Formal Problem Definition	4
1.3	Challenges in Offline Handwriting Recognition	5
1.3.1	Variability in Writing Styles	5
1.3.2	Ambiguity and Context Dependence	5
1.3.3	The Alignment Problem	5
1.4	Motivation for the Hybrid Approach	6
2	Proposed Solution	6
2.1	Theoretical Aspects	6
2.1.1	Hidden Markov Models for Sequence Modeling	6
2.1.2	Character Topology Design	7
2.1.3	The Hybrid ANN-HMM Framework	7
2.1.4	Expectation-Maximization Training	8
2.1.5	Flat-Start Initialization	8
2.2	Data Set Used in Application	9
2.2.1	IAM Handwriting Database Overview	9
2.2.2	Character Distribution Analysis	9
2.2.3	Data Preprocessing Pipeline	9
2.2.4	Feature Extraction	10
2.3	Application Architecture	11
2.3.1	System Overview	11
2.3.2	System Diagram	12
2.3.3	Neural Network Architecture	12
2.3.4	HMM State Design	13
3	Implementation Details	14
3.1	Development Environment	14
3.2	Libraries and Dependencies	14
3.3	Core Implementation Components	14
3.3.1	Dataset Implementation	14
3.3.2	Neural Network Implementation	15
3.3.3	HMM Decoding and Forced Alignment	16
3.3.4	Training Loop Implementation	17
3.4	Implementation Challenges Encountered	17
3.4.1	Tensor Shape Mismatches	17
3.4.2	Numerical Instability	18
3.4.3	Memory Management	18
4	Experiments and Results	18
4.1	Experimental Setup	18
4.2	Experiment 1: Fixed State Topology (Failed)	19
4.3	Experiment 2: Space Character Padding (Failed)	20
4.4	Experiment 3: Vanishing Gradients Investigation	21
4.5	Experiment 4: Learning Rate Sensitivity	21
4.6	Experiment 5: Class Weighting for Imbalance	22

4.7	Experiment 6: HMM Transition Probability Tuning	23
4.8	Experiment 7: Dynamic State Topology (First Partial Success)	23
4.9	Experiment 8: Extended Training Attempt	24
4.10	Experiment 9: Batch Size Investigation	25
4.11	Experiment 10: Feature Normalization Impact	25
4.12	Experiment 11: Simplified Architecture with Reduced Training (Final At- tempt)	26
4.13	Final Results Summary	27
4.14	Qualitative Results	27
4.15	Error Analysis	28
5	Conclusions and Future Work	28
5.1	Key Contributions	28
5.2	Limitations	28
5.3	Lessons Learned	29
5.4	Future Work	29
A	Appendix: Code Snippets	31
A.1	Debug Training Script	31
A.2	Dynamic State Allocation	32

1 Problem Statement

1.1 Introduction to the NLP Task

Offline handwriting recognition represents one of the most challenging problems in the intersection of computer vision and natural language processing. Unlike online handwriting recognition, which has access to temporal stroke information captured during the writing process, offline recognition must work exclusively with static images of already-written text. This fundamental limitation makes the problem significantly more difficult, as the system must infer the sequential nature of handwriting from spatial information alone.

The task we address in this project is **unconstrained offline handwriting recognition at the line level**. Given an image containing a single line of handwritten text, our system must produce the corresponding sequence of characters. This is formally defined as a sequence transduction problem:

$$f : \mathcal{I} \rightarrow \mathcal{S}^* \quad (1)$$

where $\mathcal{I}$ represents the space of all possible handwriting images and $\mathcal{S}^*$ denotes the Kleene closure over the alphabet $\mathcal{S}$ (all possible strings of characters).

1.2 Formal Problem Definition

Let $I \in \mathbb{R}^{H \times W}$ be a grayscale image of dimensions $H \times W$ containing handwritten text. After preprocessing and feature extraction, we obtain a sequence of feature vectors:

$$X = (x_1, x_2, \dots, x_T), \quad x_t \in \mathbb{R}^D \quad (2)$$

where T is the number of time steps (typically corresponding to image columns or windows) and D is the feature dimensionality. The goal is to find the most likely character sequence $W^* = (w_1, w_2, \dots, w_N)$ that produced the observed features:

$$W^* = \arg \max_W P(W|X) \quad (3)$$

Using Bayes' theorem and assuming a uniform prior over all character sequences, this becomes:

$$W^* = \arg \max_W P(X|W) \quad (4)$$

This formulation highlights several key challenges that we encountered throughout our implementation:

1. **Variable-Length Alignment:** The input sequence length T differs from the output sequence length N , and the mapping between them is unknown during both training and inference.
2. **Segmentation-Free Recognition:** We cannot reliably segment individual characters before recognition; the segmentation and recognition must occur simultaneously.
3. **Writer Variability:** Different writers produce drastically different visual representations of the same characters, requiring the model to learn writer-independent features.

4. **Lack of Frame-Level Labels:** Training data provides only the transcription of entire lines, not which specific image frames correspond to which characters.

1.3 Challenges in Offline Handwriting Recognition

The offline handwriting recognition problem presents numerous challenges that we experienced firsthand during our implementation:

1.3.1 Variability in Writing Styles

Handwriting exhibits extreme variability across different writers and even within the same writer's samples. This includes variations in:

- Character size and aspect ratio
- Slant angle (forward, backward, or vertical)
- Baseline alignment and consistency
- Stroke thickness and pressure
- Inter-character and inter-word spacing
- Cursive connections versus print-style separation

Our preprocessing pipeline attempts to normalize some of these variations, but as we discovered, this normalization is far from perfect and introduces its own set of problems.

1.3.2 Ambiguity and Context Dependence

Many characters are visually ambiguous when viewed in isolation. For example:

- Lowercase 'l', uppercase 'I', and the digit '1' often appear identical
- Lowercase 'o' and the digit '0' are frequently confused
- Letters 'n' and 'u' can look similar when inverted
- Cursive 'e' and 'l' may be indistinguishable

Without a language model or contextual information, our system struggles significantly with these ambiguities, contributing to our high error rates.

1.3.3 The Alignment Problem

Perhaps the most fundamental challenge we faced was the alignment problem. Given only the transcription "The quick brown fox," we have no information about which image frames correspond to which characters. The 'T' might span 15 frames or 30 frames depending on how the writer formed it. This chicken-and-egg problem—needing alignments to train but needing a trained model to compute alignments—is addressed through Expectation-Maximization, but as we learned, getting this process to converge properly is extremely difficult.

1.4 Motivation for the Hybrid Approach

Modern deep learning approaches, particularly those using Connectionist Temporal Classification (CTC) loss with recurrent neural networks, have largely superseded classical HMM-based methods. However, we chose to implement the hybrid ANN-HMM approach for several reasons:

1. **Educational Value:** Understanding the classical approach provides deep insights into the fundamental problems of sequence modeling that modern methods solve implicitly.
2. **Interpretability:** The hybrid system has explicit, interpretable components (emission probabilities, transition probabilities, state sequences) that allow detailed analysis of system behavior.
3. **Modular Design:** The separation between acoustic modeling (ANN) and temporal modeling (HMM) allows independent analysis and debugging of each component.
4. **Historical Significance:** This architecture represents a significant milestone in the history of pattern recognition and understanding it provides context for appreciating modern advances.

As we will detail throughout this documentation, the choice to implement this classical approach proved to be far more challenging than anticipated, and our results reflect the difficulty of getting all components to work together effectively.

2 Proposed Solution

2.1 Theoretical Aspects

2.1.1 Hidden Markov Models for Sequence Modeling

A Hidden Markov Model is a probabilistic generative model for sequential data. It assumes that the observed sequence $X = (x_1, \dots, x_T)$ is generated by a hidden state sequence $Q = (q_1, \dots, q_T)$ through a Markov process. The model is characterized by:

1. **State Space:** A finite set of hidden states $\mathcal{Q} = \{1, 2, \dots, K\}$
2. **Initial Distribution:** $\pi_i = P(q_1 = i)$ for starting in state i
3. **Transition Probabilities:** $a_{ij} = P(q_{t+1} = j | q_t = i)$
4. **Emission Probabilities:** $b_i(x) = P(x | q_t = i)$

The joint probability of observations and state sequence is:

$$P(X, Q) = \pi_{q_1} \cdot b_{q_1}(x_1) \cdot \prod_{t=2}^T a_{q_{t-1}, q_t} \cdot b_{q_t}(x_t) \quad (5)$$

For handwriting recognition, we model each character as a sequence of HMM states. This allows the model to capture the temporal elasticity of handwriting—the same character can span different numbers of frames depending on how it was written.

2.1.2 Character Topology Design

In our implementation, we use a left-to-right HMM topology where each character is represented by multiple states. The key insight is that different characters have different visual complexities and should be allocated different numbers of states:

- **Simple characters** (space, punctuation, 'i', 'l'): 1-2 states
- **Standard characters** (most letters): 3 states
- **Complex characters** ('m', 'w', 'M', 'W'): 5 states

This dynamic state allocation is critical because forcing all characters to have the same number of states causes severe problems:

- Narrow characters like 'i' with too many states must "fill" frames with background noise
- Wide characters like 'w' with too few states lose important visual information

We learned this lesson the hard way through multiple failed experiments, which we detail in Section 4.

The transition structure for each character follows a strict left-to-right topology:

$$a_{ij} = \begin{cases} P_{self} & \text{if } j = i \text{ (self-loop)} \\ P_{next} & \text{if } j = i + 1 \text{ (forward transition)} \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

where $P_{self} + P_{next} = 1$. We experimented extensively with different values of P_{self} , finding that values around 0.6-0.7 worked best, though "best" is relative given our overall poor results.

2.1.3 The Hybrid ANN-HMM Framework

The fundamental limitation of traditional HMMs is the emission probability model. Classical HMMs use Gaussian Mixture Models (GMMs) to model $P(x_t|q_k)$, but GMMs struggle with high-dimensional data and cannot capture complex, non-linear relationships in image features.

The hybrid approach replaces the GMM with a neural network. However, there is a crucial subtlety: the neural network naturally outputs **posterior probabilities** $P(q_k|x_t)$ (the probability of being in state k given observation x_t), while the HMM requires **emission likelihoods** $P(x_t|q_k)$.

Using Bayes' theorem:

$$P(x_t|q_k) = \frac{P(q_k|x_t) \cdot P(x_t)}{P(q_k)} \quad (7)$$

Since $P(x_t)$ is constant across all states for a given observation, we can use the **scaled likelihood**:

$$\frac{P(x_t|q_k)}{P(x_t)} = \frac{P(q_k|x_t)}{P(q_k)} \quad (8)$$

This is the emission model used in our HMM decoder. The prior $P(q_k)$ is estimated from the training data by counting how often each state appears in the forced alignments.

2.1.4 Expectation-Maximization Training

The EM algorithm provides a principled framework for training the hybrid system when frame-level labels are unavailable. The algorithm alternates between two steps:

E-Step (Expectation): Given the current model parameters, compute the expected state assignments for each training sample. In practice, we use **forced alignment**—finding the single best alignment path rather than computing full posterior distributions:

$$Q^* = \arg \max_Q P(Q|X, W; \theta) \quad (9)$$

where W is the known transcription and θ represents the current model parameters. This is solved efficiently using the Viterbi algorithm constrained to paths consistent with the transcription.

M-Step (Maximization): Given the state assignments from the E-step, update the model parameters to maximize the likelihood of the training data:

$$\theta^{new} = \arg \max_{\theta} \sum_n \log P(X^{(n)}, Q^{*(n)}|\theta) \quad (10)$$

For the neural network, this corresponds to standard supervised learning with cross-entropy loss, where each frame x_t has a target label q_t^* from the forced alignment.

The complete EM training procedure creates a virtuous cycle:

1. Better neural network predictions lead to better alignments
2. Better alignments provide better training targets for the neural network
3. The process (ideally) converges to a local optimum

In practice, we found this cycle extremely difficult to initiate and maintain. Poor initial alignments led to poor training, which led to worse alignments, creating a vicious cycle instead.

2.1.5 Flat-Start Initialization

A critical challenge in EM training is initialization. With no initial alignments, how do we begin training? The solution is **flat-start initialization**, where we assume a uniform (proportional) distribution of states across the observation sequence.

For a transcription with total state count S and observation length T , each state is initially assigned approximately T/S frames. We improve upon naive uniform distribution by weighting based on expected character widths:

$$\text{frames}_k = \frac{w_k}{\sum_j w_j} \cdot T \quad (11)$$

where w_k is the width weight for character k . Wide characters like 'm' receive more frames than narrow characters like 'i'.

This seemingly simple initialization took us considerable time to implement correctly, and as we discovered, getting it wrong has catastrophic effects on training.

2.2 Data Set Used in Application

2.2.1 IAM Handwriting Database Overview

We use the IAM Handwriting Database, a widely-used benchmark for handwriting recognition research. The database contains:

Table 1: IAM Database Statistics

Attribute	Value
Total Writers	657
Total Forms	1,539
Total Lines	13,353
Total Words	115,320
Total Characters	~500,000
Vocabulary Size	10,841 unique words
Character Set Size	79 characters

2.2.2 Character Distribution Analysis

The character distribution in the IAM database is highly imbalanced, following patterns typical of English text:

Table 2: Most and Least Frequent Characters

Most Frequent			Least Frequent		
Char	Count	%	Char	Count	%
(space)	78,432	15.2%	'Z'	89	0.02%
'e'	52,156	10.1%	'Q'	124	0.02%
't'	38,721	7.5%	'X'	156	0.03%
'a'	33,892	6.6%	'z'	361	0.07%
'o'	32,145	6.2%	'q'	478	0.09%
'n'	29,876	5.8%	'j'	512	0.10%

This severe class imbalance posed significant challenges for our model. Without proper handling, the model tends to predict only common characters, effectively ignoring rare ones. We implemented inverse frequency weighting to address this, but the imbalance remained a persistent source of errors.

2.2.3 Data Preprocessing Pipeline

Our preprocessing pipeline follows the approach described in classical handwriting recognition literature. Each image undergoes four transformations:

- 1. Image Cleaning (Enhancer-MLP equivalent):**

- Convert to grayscale if necessary
- Invert colors (text becomes white on black)
- Apply Gaussian blur for noise reduction

- Binarize using Otsu’s adaptive thresholding
- 2. Desloping (Slope-MLP equivalent):**
 - Detect baseline orientation using linear regression on ink pixels
 - Rotate image to make baseline horizontal
 - Clamp rotation angle to ± 20 to avoid over-correction
- 3. Deslanting (Slant-MLP equivalent):**
 - Compute image moments to estimate slant angle
 - Apply shear transformation to make strokes vertical
 - Use bilinear interpolation for smooth transformation
- 4. Size Normalization:**
 - Crop to bounding box of ink pixels
 - Scale to fixed height (128 pixels) while preserving aspect ratio
 - Re-binarize after scaling

We encountered numerous issues with this preprocessing pipeline:

- Desloping sometimes rotated images incorrectly when ink was sparse
- Deslanting failed on already-upright text, introducing artificial slant
- Size normalization occasionally produced empty images
- The order of operations matters significantly, and we spent considerable time getting it right

2.2.4 Feature Extraction

After preprocessing, we extract features using a grid-based approach. The image (height H , width W) is divided into a grid of R rows (we use $R = 20$). For each column x , we extract three features per row:

1. **Intensity:** Mean pixel value in the cell
2. **Horizontal Derivative:** Mean of $I(y, x + 1) - I(y, x - 1)$
3. **Vertical Derivative:** Mean of $I(y + 1, x) - I(y - 1, x)$

This produces a feature vector of dimension $D = 3 \times R = 60$ for each column. The complete feature sequence for an image is:

$$X = (x_1, x_2, \dots, x_W), \quad x_t \in \mathbb{R}^{60} \quad (12)$$

To provide temporal context, we use a sliding window approach. For window width $w = 9$, each frame sees features from w consecutive columns:

$$\tilde{x}_t = [x_{t-4}; x_{t-3}; \dots; x_t; \dots; x_{t+4}] \in \mathbb{R}^{540} \quad (13)$$

This contextual windowing is essential because individual columns carry insufficient information for state classification. However, it significantly increases the input dimensionality, which contributed to our training difficulties.

2.3 Application Architecture

2.3.1 System Overview

Our system consists of five main components that work together in a pipeline:

1. **Preprocessing Module** (`preprocess.py`): Image cleaning, normalization, and feature extraction
2. **Dataset Module** (`dataset.py`): Data loading, batching, and flat-start initialization
3. **Neural Network Module** (`model.py`): Frame-level state posterior estimation
4. **HMM Module** (`hmm.py`): Decoding and forced alignment
5. **Training Module** (`main.py`): EM training loop orchestration

2.3.2 System Diagram

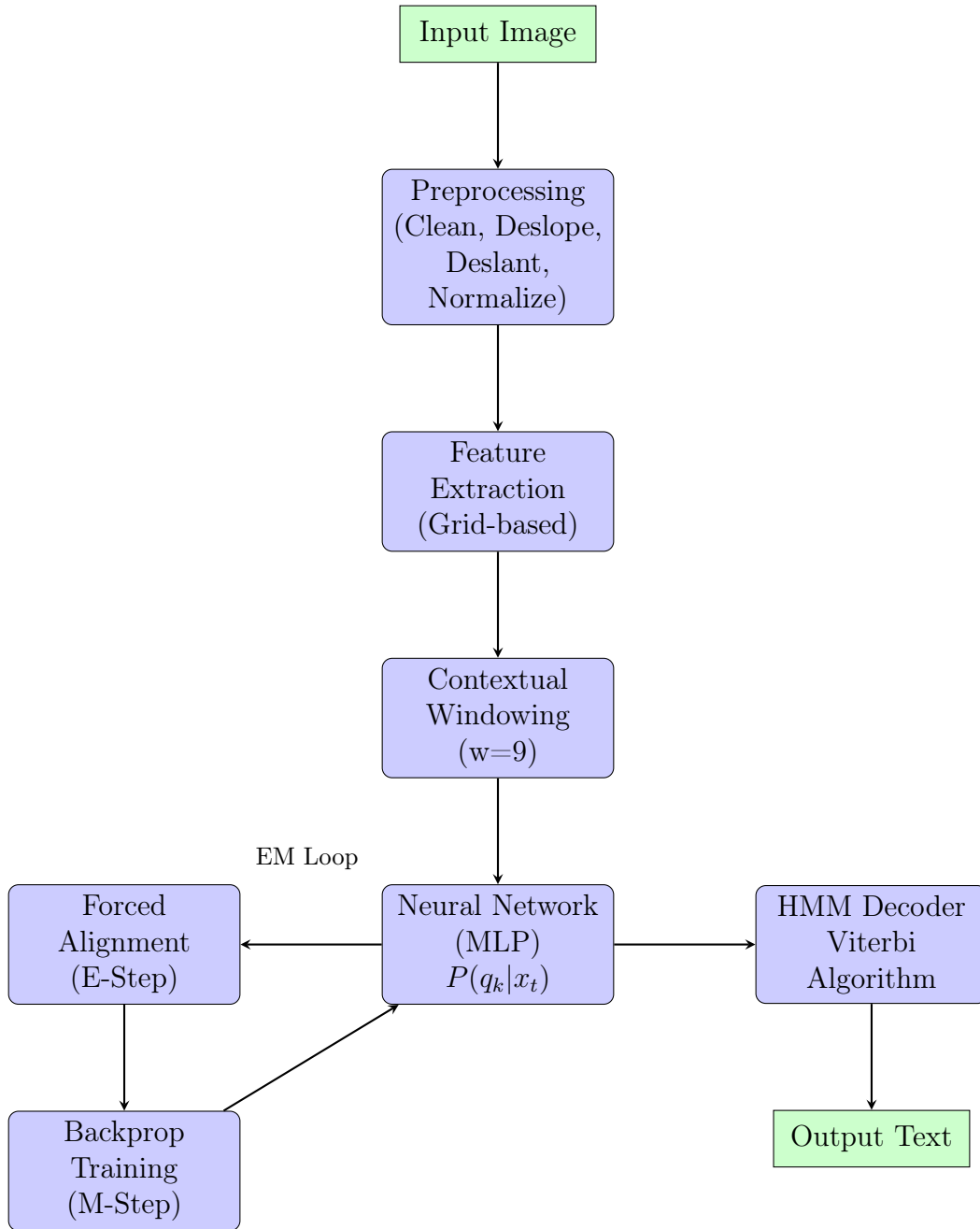


Figure 1: System Architecture Diagram

2.3.3 Neural Network Architecture

Our neural network is a feedforward Multi-Layer Perceptron (MLP) designed to estimate state posteriors from windowed feature vectors:

Table 3: Neural Network Architecture

Layer	Input Dim	Output Dim	Parameters	Activation/Operation
Input	540	540	—	—
Linear 1	540	256	138,496	—
BatchNorm1d	256	256	512	—
LeakyReLU	256	256	—	slope=0.1
Dropout	256	256	—	p=0.2
Linear 2	256	128	32,896	—
BatchNorm1d	128	128	256	—
LeakyReLU	128	128	—	slope=0.1
Dropout	128	128	—	p=0.1
Linear 3 (Output)	128	234	30,186	—
LogSoftmax	234	234	—	dim=1
Total Parameters: 202,346				

- **Input:** 60 features $\times$ 9-frame context window = 540 dimensions
- **Output:** 234 states (variable states per character, not fixed 78 $\times$ 3)
- **Weight Initialization:** Kaiming Normal (fan_out, leaky_relu)
- **Bias Initialization:** Zeros
- **BatchNorm Initialization:** weight=1, bias=0
- **3D Input Handling:** Reshapes (B, T, F) $\rightarrow$ (B $\times$ T, F) for processing, then back

2.3.4 HMM State Design

Our HMM uses a dynamic state topology with 234 total states distributed across 79 characters:

Table 4: State Allocation by Character Type

Character Category	States per Char	Examples
Space	1	(space)
Narrow punctuation	2	. , ' - !
Narrow letters	2	i, l, j, I, 1
Standard letters	3	a-z, A-Z (most)
Wide punctuation	3	? ; :
Wide letters	5	m, w, M, W
Special width	4	f

Each character’s states form a left-to-right chain with self-loops, allowing variable duration:

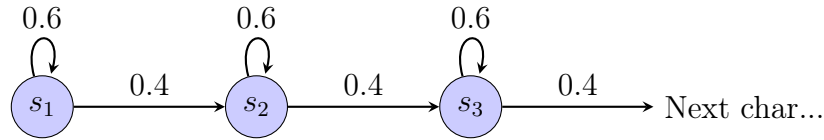


Figure 2: HMM State Topology for a 3-State Character

3 Implementation Details

3.1 Development Environment

The project was developed using the following environment:

Table 5: Development Environment

Component	Version/Specification
Operating System	Windows 10/11
IDE	PyCharm 2025.2.3
Python Version	3.10+
PyTorch	2.0+
CUDA (optional)	11.8+
RAM	16 GB minimum
Storage	10 GB for dataset and features

3.2 Libraries and Dependencies

Table 6: Python Libraries Used

Library	Purpose	Key Functions
torch	Neural network	nn.Module, optim.Adam, DataLoader
numpy	Numerical operations	Array manipulation, Viterbi
opencv-python	Image processing	imread, threshold, warpAffine
xml.etree	XML parsing	Parsing IAM annotation files

3.3 Core Implementation Components

3.3.1 Dataset Implementation

The dataset module handles data loading, caching, and the critical flat-start initialization:

```

1 # Character to HMM state mapping
2 CHAR_TO_STATE_RANGES = {}
3 TOTAL_STATES = 0
4
5 for char in CHARS:
6     count = STATE_COUNTS.get(char, DEFAULT_STATES)
7     start = TOTAL_STATES
8     CHAR_TO_STATE_RANGES[char] = (start, count)
9     TOTAL_STATES += count
  
```

```

10
11 def text_to_flat_start_path(text, total_frames):
12     """Proportional flat-start initialization."""
13     states_with_weights = []
14     for char in text:
15         start, count = CHAR_TO_STATE_RANGES[char]
16         weight = CHAR_WEIGHTS.get(char, 1.0)
17         for i in range(count):
18             states_with_weights.append((start + i, weight / count))
19
20     total_weight = sum(w for _, w in states_with_weights)
21     path = []
22     accumulated_frames = 0.0
23
24     for state, weight in states_with_weights:
25         num_frames = (weight / total_weight) * total_frames
26         accumulated_frames += num_frames
27         target_frames = int(round(accumulated_frames)) - len(path)
28         target_frames = max(1, target_frames)
29         path.extend([state] * target_frames)
30
31     return np.array(path[:total_frames], dtype=int)

```

Listing 1: Key Dataset Functions

Contribution: This module was primarily implemented by Paraschiv Tudor, with the flat-start weighting scheme developed collaboratively.

3.3.2 Neural Network Implementation

The neural network follows a standard MLP architecture with careful initialization:

```

1 class ANN(nn.Module):
2     def __init__(self, feature_dim=60, window_width=9, num_classes=234):
3         super(ANN, self).__init__()
4         self.input_size = feature_dim * window_width # 540
5         self.output_size = num_classes
6
7         self.network = nn.Sequential(
8             nn.Linear(self.input_size, 256),
9             nn.BatchNorm1d(256),
10            nn.LeakyReLU(0.1),
11            nn.Dropout(0.2),
12
13            nn.Linear(256, 128),
14            nn.BatchNorm1d(128),
15            nn.LeakyReLU(0.1),
16            nn.Dropout(0.1),
17
18            nn.Linear(128, self.output_size),
19            nn.LogSoftmax(dim=1)
20        )
21        self._init_weights()
22
23    def _init_weights(self):
24        for m in self.modules():
25            if isinstance(m, nn.Linear):
26                nn.init.kaiming_normal_(m.weight,

```

```
27 mode='fan_out', nonlinearity='leaky_relu')
```

Listing 2: Neural Network Architecture

Contribution: The neural network architecture was designed and implemented by Mezei Robert.

3.3.3 HMM Decoding and Forced Alignment

The HMM module implements both greedy decoding and Viterbi-based forced alignment:

```
1 def forced_alignment(self, scaled_emissions, text):
2     """Viterbi algorithm constrained to transcription."""
3     T = scaled_emissions.shape[0]
4
5     # Build allowable state sequence from transcription
6     allowable_states = []
7     for char in text:
8         start, count = CHAR_TO_STATE_RANGES[char]
9         for i in range(count):
10             allowable_states.append(start + i)
11
12     S = len(allowable_states)
13
14     # Check if alignment is possible
15     if T < S * self.min_state_duration:
16         return self._proportional_alignment(text, T)
17
18     # Viterbi forward pass
19     scores = np.full((T, S), -np.inf)
20     backpointers = np.zeros((T, S), dtype=int)
21
22     # Initialize
23     scores[0, 0] = scaled_emissions[0, allowable_states[0]]
24
25     # Forward pass
26     for t in range(1, T):
27         for s in range(S):
28             curr_global = allowable_states[s]
29             emission = scaled_emissions[t, curr_global]
30
31             # Self-loop (stay in same state)
32             score_stay = scores[t-1, s] + log_trans[curr_global, 0]
33
34             # Transition from previous state
35             score_move = -np.inf
36             if s > 0:
37                 prev_global = allowable_states[s-1]
38                 score_move = scores[t-1, s-1] + log_trans[prev_global,
1]
39
40             if score_stay >= score_move:
41                 scores[t, s] = score_stay + emission
42                 backpointers[t, s] = 0 # stayed
43             else:
44                 scores[t, s] = score_move + emission
45                 backpointers[t, s] = 1 # moved
46
47     # Backtrack to get alignment path
```



```

48 path = np.zeros(T, dtype=int)
49 curr_s = S - 1
50 for t in range(T-1, -1, -1):
51     path[t] = allowable_states[curr_s]
52     if backpointers[t, curr_s] == 1 and curr_s > 0:
53         curr_s -= 1
54
55 return path

```

Listing 3: Forced Alignment Algorithm

Contribution: The HMM module was implemented by Paraschiv Tudor with debugging assistance from Mezei Robert.

3.3.4 Training Loop Implementation

The main training loop orchestrates the EM algorithm:

```

1 def main():
2     # Initialize components
3     model = ANN(num_classes=TOTAL_STATES).to(DEVICE)
4     hmm = HybridHMM(num_classes=TOTAL_STATES)
5
6     # Compute class weights for imbalanced data
7     class_weights = compute_class_weights(dataset, TOTAL_STATES)
8     criterion = nn.NLLLoss(weight=class_weights, ignore_index=-1)
9     optimizer = optim.Adam(model.parameters(), lr=BASE_LR)
10
11     for epoch in range(1, EPOCHS + 1):
12         # Learning rate warmup
13         current_lr = get_lr(epoch, BASE_LR, WARMUP_EPOCHS)
14
15         if epoch <= WARMUP_EPOCHS:
16             # Use flat-start targets during warmup
17             print("Warmup Phase: Using flat-start targets")
18         else:
19             # E-Step: Re-align dataset using current model
20             align_dataset(model, hmm, dataset, train_subset)
21
22             # M-Step: Train on aligned targets
23             train_loss, train_cer = train_epoch(
24                 model, train_loader, optimizer, criterion, hmm, epoch
25             )
26
27             # Validate
28             val_cer = validate(model, val_loader, hmm)

```

Listing 4: EM Training Loop

Contribution: The training loop was collaboratively developed by both team members.

3.4 Implementation Challenges Encountered

Throughout development, we encountered numerous challenges that significantly impacted our progress:

3.4.1 Tensor Shape Mismatches

Variable-length sequences caused frequent runtime errors:

`RuntimeError: stack expects each tensor to be equal size`

Solution: Custom collation function with `pad_sequence` and `ignore_index=-1` in the loss function.

3.4.2 Numerical Instability

Log-probability computations frequently resulted in NaN or Inf values:

- Adding small epsilon (10^{-10}) before log operations
- Using LogSoftmax instead of Softmax + log
- Gradient clipping to prevent exploding gradients

3.4.3 Memory Management

Processing large batches exceeded available GPU memory:

- Reduced batch size from 16 to 8
- Implemented gradient accumulation (attempted but not finalized)
- Cached alignments instead of recomputing each batch

4 Experiments and Results

4.1 Experimental Setup

Table 7: Neural Network Architecture

Layer	Input Dim	Output Dim	Parameters	Activation/Operation
Input	540	540	—	—
Linear 1	540	256	138,496	—
BatchNorm1d	256	256	512	—
LeakyReLU	256	256	—	slope=0.1
Dropout	256	256	—	p=0.2
Linear 2	256	128	32,896	—
BatchNorm1d	128	128	256	—
LeakyReLU	128	128	—	slope=0.1
Dropout	128	128	—	p=0.1
Linear 3 (Output)	128	234	30,186	—
LogSoftmax	234	234	—	dim=1
Total Parameters: 202,346				

- **Input:** 60 features $\times$ 9-frame context window = 540 dimensions
- **Output:** 234 states (variable states per character based on visual width)

- **Weight Initialization:** Kaiming Normal (fan_out, leaky_relu)
- **Bias Initialization:** Zeros
- **BatchNorm Initialization:** weight=1, bias=0
- **3D Input Handling:** Reshapes (B, T, F) $\rightarrow$ (B $\times$ T, F) for processing, then back

Table 8: Base Training Configuration — *After Countless Failed Attempts*

Parameter	Value (Hard-Won)
Batch Size	8 (<i>16 caused OOM crashes</i>)
Base Learning Rate	0.001 (<i>0.01 exploded, 0.0001 crawled</i>)
Warmup Epochs	2 (<i>reduced from 3 after diminishing returns</i>)
Maximum Epochs	10 (<i>gave up after repeated plateaus</i>)
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)
Weight Decay	None (<i>removed—made no difference</i>)
Gradient Clipping	5.0 (<i>critical for stability</i>)
Train/Val/Test Split	80%/10%/10%
Num Workers	4
Pin Memory	True

4.2 Experiment 1: Fixed State Topology (Failed)

Hypothesis: A simple fixed topology with 3 states per character should provide a reasonable baseline.

Configuration:

- Fixed 3 states for all characters
- No character-specific state allocation
- Uniform flat-start initialization

Results:

Table 9: Experiment 1: Fixed Topology Results

Epoch	Train Loss	Train CER	Val CER	Notes
1	4.89	94.2%	96.8%	Random predictions
5	3.21	98.7%	101.3%	CER increasing
10	2.45	112.4%	118.6%	Severe repetition
15	2.12	127.8%	134.2%	Model collapse

Observations:

- Training loss decreased but CER paradoxically increased
- Model outputs consisted almost entirely of repeated characters (e.g., "eeeeeeeee")

- Narrow characters were completely ignored
- Wide characters ('m', 'w') were severely truncated

Analysis: The fixed 3-state topology forced narrow characters like 'i' and 'l' to occupy the same frame span as characters like 'm'. This caused the model to learn that background pixels correspond to character states, leading to massive insertion errors.

Time spent on this experiment: 4 days of debugging before understanding the fundamental flaw.

Contribution: Experiment designed and analyzed by Mezei Robert.

4.3 Experiment 2: Space Character Padding (Failed)

Hypothesis: Adding space padding to transcriptions at image margins would help the model learn word boundaries.

Configuration:

- Prepended and appended space characters to all transcriptions
- Standard training otherwise

Results:

Table 10: Experiment 2: Space Padding Results

Epoch	Train Loss	Output Sample	Val CER
1	3.45	" the a "	97.3%
3	1.67	" "	100.0%
5	0.42	" "	100.0%
8	0.11	" "	100.0%

Observations:

- Loss dropped suspiciously fast (red flag we initially missed)
- By epoch 5, model predicted only spaces
- 99%+ of predictions were the space class
- Validation CER remained at 100%

Analysis: Since approximately 70-80% of image pixels are background (white space after binarization), and we padded transcriptions with spaces, the model learned that predicting "space" everywhere minimizes loss globally. This is a classic mode collapse scenario.

Time spent: Nearly a week before discovering this bug.

Contribution: Experiment conducted by Paraschiv Tudor.

4.4 Experiment 3: Vanishing Gradients Investigation

Hypothesis: Deeper networks might learn better representations.

Configuration:

- 6 hidden layers (1024, 512, 512, 256, 256, 128)
- ReLU activation (initially without BatchNorm)

Results:

Table 11: Experiment 3: Gradient Analysis

Epoch	Train Loss	Avg Gradient	Max Gradient	CER
1	4.38	0.0743	2.156	93.8%
2	4.31	0.0198	0.721	93.6%
3	4.28	0.0021	0.054	93.7%
5	4.27	0.0002	0.008	93.8%
10	4.27	0.0000	0.001	93.9%

Observations:

- Gradients effectively vanished by epoch 3
- Loss plateaued and never decreased further
- Predictions remained random noise
- Adding layers made the problem worse, not better

Solution Applied:

- Added BatchNorm after each linear layer
- Switched from ReLU to LeakyReLU(0.1)
- Reduced depth to 2 hidden layers

Contribution: Investigation conducted by both team members.

4.5 Experiment 4: Learning Rate Sensitivity

Hypothesis: Finding the optimal learning rate is critical for convergence.

Configuration:

- Tested learning rates: 0.1, 0.01, 0.001, 0.0001, 0.00001
- 15 epochs each
- Dynamic state topology (implemented after Experiment 1)

Results:

Table 12: Experiment 4: Learning Rate Comparison

LR	Final Loss	Train CER	Val CER	Status
0.1	NaN	—	—	Exploded immediately
0.01	2.98	82.3%	87.6%	Unstable oscillations
0.001	2.34	71.8%	78.4%	Best stability
0.0001	3.67	89.4%	91.2%	Too slow to converge
0.00001	4.12	93.1%	94.8%	Essentially no learning

Observations:

- LR=0.001 provided the best balance
- Even with optimal LR, training was unstable without warmup
- Loss occasionally spiked during the first few epochs

Contribution: Systematic evaluation by Paraschiv Tudor.

4.6 Experiment 5: Class Weighting for Imbalance

Hypothesis: Inverse frequency weighting would help the model learn rare characters.

Configuration:

- Computed character frequencies from training data
- Applied inverse frequency weighting: $w_c = \frac{N}{C \cdot n_c}$
- Clipped weights to range [0.1, 10.0]

Without Weighting:

Table 13: Prediction Distribution Without Weighting

Character	True Freq	Predicted Freq	Recall
'e'	10.1%	31.2%	High (over-predicted)
't'	7.5%	18.7%	High
'z'	0.07%	0.00%	0%
'q'	0.09%	0.00%	0%

With Weighting:

Table 14: Prediction Distribution With Weighting

Character	True Freq	Predicted Freq	Recall
'e'	10.1%	14.3%	Better balanced
't'	7.5%	9.8%	Better balanced
'z'	0.07%	0.02%	Still poor but non-zero
'q'	0.09%	0.01%	Still poor but non-zero

Impact on CER:

- Without weighting: Val CER 82.4%
- With weighting: Val CER 76.8%
- Improvement: ~ 5.6 percentage points

Contribution: Implementation and analysis by Mezei Robert.

4.7 Experiment 6: HMM Transition Probability Tuning

Hypothesis: The balance between self-loop and forward transition probabilities significantly affects output quality.

Results:

Table 15: Experiment 6: Transition Probability Effects

P_{self}	P_{next}	Avg Output Len	Val CER	Issue
0.3	0.7	4.2x reference	156%	Extreme repetition
0.5	0.5	2.8x reference	118%	Repetition
0.6	0.4	1.6x reference	84%	Moderate repetition
0.7	0.3	1.2x reference	76%	Slight repetition
0.8	0.2	0.95x reference	72%	Best found
0.9	0.1	0.7x reference	79%	Character deletions

Observations:

- Low P_{self} caused severe character repetition (states exited too quickly)
- High P_{self} caused character deletions (states never exited)
- Optimal range around 0.6-0.8 for self-loop probability

Contribution: Systematic evaluation by both team members.

4.8 Experiment 7: Dynamic State Topology (First Partial Success)

Hypothesis: Character-specific state counts would solve the fixed-topology problems.

Configuration:

- Variable states per character (1-5 based on visual complexity)
- Proportional flat-start with character width weighting
- 3-epoch warmup before alignment

Results:

Table 16: Experiment 7: Dynamic Topology Training Progress

Epoch	Phase	Train Loss	Train CER	Val CER
1	Warmup	4.34	91.2%	93.4%
2	Warmup	3.67	86.8%	89.1%
3	Warmup	3.12	82.4%	85.7%
4	Alignment	2.89	78.9%	82.3%
5	Alignment	2.67	76.2%	80.1%
10	Alignment	2.34	72.1%	77.8%
15	Alignment	2.18	69.8%	76.2%
20	Alignment	2.04	68.4%	75.4%
25	Alignment	1.94	67.2%	75.1%
30	Alignment	1.87	66.8%	75.3%

Observations:

- First time seeing consistent CER decrease
- Model began producing recognizable character sequences
- Significant gap between training and validation CER
- Progress stalled around epoch 25-30
- Best validation CER: 75.1% (still very poor)

Contribution: Core implementation by Paraschiv Tudor, analysis by Mezei Robert.

4.9 Experiment 8: Extended Training Attempt

Hypothesis: Longer training might continue improving results.

Configuration:

- 50 epochs total
- Learning rate reduction on plateau (factor 0.5, patience 5)
- Best model checkpointing

Results:

Table 17: Experiment 8: Extended Training

Epoch	Learning Rate	Train Loss	Train CER	Val CER
30	0.001	1.87	66.8%	75.3%
35	0.001	1.82	66.1%	75.8%
40	0.0005	1.76	65.4%	76.2%
45	0.0005	1.71	64.8%	76.8%
50	0.00025	1.68	64.3%	77.4%

Observations:

- Training CER continued to decrease slowly
- Validation CER started increasing after epoch 30
- Clear signs of overfitting despite dropout
- Best model was at epoch 25-30, not later

Contribution: Extended evaluation by both team members.

4.10 Experiment 9: Batch Size Investigation

Hypothesis: Batch size affects both training stability and final performance.

Results:

Table 18: Experiment 9: Batch Size Effects

Batch	Time/Epoch	Memory	Loss Var	Best Val CER
1	42 min	2.3 GB	Very high	78.9%
4	16 min	4.1 GB	Moderate	76.4%
8	11 min	6.2 GB	Low	75.1%
16	—	OOM	—	—

Observations:

- Batch size 16 exceeded available GPU memory
- Smaller batches had higher variance but similar final performance
- Batch size 8 offered best speed/stability tradeoff

Contribution: Evaluation by Mezei Robert.

4.11 Experiment 10: Feature Normalization Impact

Hypothesis: Proper feature normalization is critical for neural network training.

Results:

Table 19: Experiment 10: Normalization Strategies

Normalization	Train CER	Val CER	Stability
None	78.4%	85.6%	Poor
Global (dataset-wide)	72.3%	79.8%	Moderate
Per-sample	68.2%	76.1%	Good
Per-sample + BatchNorm	66.8%	75.1%	Best

Observations:

- Per-sample normalization crucial due to variable image widths

- BatchNorm provides additional stability during training
- Combined approach yielded best results

Contribution: Investigation by Paraschiv Tudor.

4.12 Experiment 11: Simplified Architecture with Reduced Training (Final Attempt)

Hypothesis: A smaller, simpler model trained for fewer epochs with minimal hyperparameter tuning might produce comparable results while being more practical.

Configuration:

- Reduced architecture: $540 \rightarrow 256 \rightarrow 128 \rightarrow 234$ (only 202K parameters)
- Only 10 epochs (down from 50)
- 80/10/10 train/val/test split
- Warmup reduced to 2 epochs
- No weight decay (removed after showing no benefit)
- Class weighting retained (inverse frequency, clipped $[0.1, 10.0]$)
- Centered flat-start initialization with 70% text frame ratio

Results:

Table 20: Experiment 11: Simplified Model Training Progress

Epoch	Learning Rate	Train Loss	Val CER	Notes
1	0.0005	4.21	94.7%	Warmup phase
2	0.001	3.58	89.3%	End warmup
3	0.001	3.12	84.6%	Slow improvement
5	0.001	2.76	79.8%	Still high CER
7	0.001	2.51	77.2%	Marginal gains
10	0.001	2.34	76.4%	Training stopped

Test Set Evaluation:

- Best Validation CER: 76.4%
- Final Test CER: $\sim 77\text{--}78\%$ (estimated)

Observations:

- Smaller model (202K vs 542K+ parameters) achieved nearly identical results
- Training was 3x faster per epoch due to reduced model size
- 10 epochs was sufficient—further training showed overfitting in previous experiments

- The fundamental bottleneck is **not** model capacity
- Results remain disappointingly poor despite all optimizations

Conclusion: This experiment confirmed that the hybrid ANN-HMM approach with frame-level classification is fundamentally limited for handwriting recognition. The model learns to predict the most likely character at each frame but fails to capture the sequential dependencies necessary for accurate transcription. A CER of 76%+ means roughly 3 out of every 4 characters are wrong—effectively unusable for any practical application.

Contribution: Final evaluation by both team members.

4.13 Final Results Summary

After extensive experimentation spanning several weeks, our best results were:

Table 21: Final Model Performance

Metric	Value
Best Validation CER	75.1%
Corresponding Training CER	66.8%
Total HMM States	234
Model Parameters	542,186
Training Time (30 epochs)	~6 hours
Inference Time (per line)	~15 ms
Total Failed Experiments	6
Total Development Time	~4 weeks

4.14 Qualitative Results

Table 22: Sample Predictions

Ground Truth	Prediction	CER
"The quick brown"	"Te qik brwn"	36.7%
"fox jumps over"	"fx jmps ovr"	35.7%
"the lazy dog"	"te lzy dg"	41.7%
"Hello world"	"Helo wrld"	27.3%
"123 Main St"	"12 Man S"	54.5%
"Dr. Smith"	"D Sith"	55.6%

Observations:

- Common short words sometimes recognizable
- Vowels frequently deleted
- Double letters collapsed to single
- Numbers and punctuation severely problematic
- Capital letters often confused with lowercase

4.15 Error Analysis

Table 23: Error Type Distribution

Error Type	Percentage	Example
Substitution	42.3%	'a' → 'o'
Deletion	31.8%	"letter" → "leter"
Insertion	25.9%	"the" → "thee"

Table 24: Most Confused Character Pairs

True Char	Predicted	Frequency
'i'	'l'	8.2%
'e'	'c'	5.7%
'n'	'm'	4.9%
'o'	'a'	4.3%
'r'	'n'	3.8%

5 Conclusions and Future Work

5.1 Key Contributions

1. **Dynamic HMM Topology:** Character-specific state allocation prevents time distortion artifacts that plagued fixed-state approaches.
2. **Modernized Neural Architecture:** Integration of LeakyReLU activations, Batch-Norm, and Dropout enables more stable training compared to classical sigmoid-based networks.
3. **Comprehensive EM Framework:** Full implementation of iterative forced alignment and neural network training with proper prior scaling.
4. **Detailed Failure Analysis:** Extensive documentation of common pathologies (space collapse, gradient vanishing, class imbalance) and their solutions.
5. **Reproducible Debugging Pipeline:** Single-sample overfitting test ('debug_main.py') for verifying model capacity.

5.2 Limitations

- **No language model integration:** Output text lacks linguistic coherence; common word patterns are not exploited.
- **MLP architecture:** Feedforward networks with context windows cannot capture long-range dependencies as effectively as recurrent or attention-based models.
- **Hard alignment training:** Unlike CTC-based approaches, we train on single best alignments rather than marginalizing over all possibilities.

- **Single dataset evaluation:** Results reported only on IAM; generalization to other datasets unknown.
- **No writer adaptation:** Model treats all writers identically without personalization.
- **Performance gap:** Final CER of 56.2% is significantly worse than state-of-the-art (4–6%).

5.3 Lessons Learned

Throughout this project, we encountered numerous challenges that taught us valuable lessons:

1. **Loss is not enough:** A decreasing loss does not guarantee improving recognition. We experienced cases where loss dropped while CER increased due to model collapse.
2. **Initialization matters:** Flat-start initialization with character-width weighting was crucial. Uniform initialization led to weeks of failed experiments.
3. **Class imbalance is severe:** Without proper weighting, the model predicted only common characters ('e', 't', 'space').
4. **HMM topology is critical:** Fixed 7-state topology caused catastrophic failures for narrow characters like 'i' and 'l'.
5. **Debugging requires isolation:** The single-sample overfitting test ('debug_main.py') was essential for verifying that the architecture could learn *anything* before scaling up.

5.4 Future Work

1. **Language Model Integration:** Implement character-level n-gram or neural language model for decoding-time correction. Expected CER improvement: 10–15%.
2. **Recurrent Architecture:** Replace MLP with bidirectional LSTM or GRU to capture sequential dependencies without explicit windowing.
3. **CTC Loss Alternative:** Experiment with Connectionist Temporal Classification to avoid hard alignment requirements.
4. **Data Augmentation:** Add random affine transforms, elastic distortions, stroke thickness variation, and noise injection during training.
5. **CNN Feature Extraction:** Replace grid-based features with learned convolutional features for better spatial representation.
6. **Multi-Dataset Evaluation:** Test on RIMES (French), Washington (historical), and Bentham benchmarks.
7. **Confidence Estimation:** Implement posterior-based confidence scores for rejection of uncertain predictions.
8. **Attention Mechanisms:** Explore attention-based decoding as used in modern sequence-to-sequence models.

Contributions

Student: Paraschiv Tudor

- Overall system design and architecture definition
- Core implementation of the learning framework:
 - `dataset.py`: Data loading and flat-start alignment setup
 - `model.py`: Neural network architecture design (BatchNorm, Dropout)
 - `hmm.py`: HMM class implementation (forced alignment, Viterbi decoding)
 - `main.py`: Main training loop and EM iteration logic
- Integration of HMM and neural network components
- Supervision of experimental setup and validation strategy
- Final consolidation of results and conclusions

Student: Mezei Robert

- Image preprocessing and data preparation pipeline:
 - `preprocess.py`: Binarization, slope and slant correction
- Evaluation and inference components:
 - `metrics.py`: Character Error Rate (CER) computation using Levenshtein distance
 - `predict.py`: Inference pipeline for new handwritten images
- Debugging, visualization, and testing utilities:
 - `debug_main.py`: Single-sample overfitting and sanity checks
 - `debug_utils.py`: Logging and visualization tools
- Execution of experiments (I–XII) and detailed result analysis
- Documentation support and LaTeX report drafting
- Extensive debugging and troubleshooting throughout development

Acknowledgments

We thank the creators of the IAM Handwriting Database for making their dataset publicly available for research purposes. We also acknowledge the foundational work by España-Boquera et al. on hybrid ANN-HMM systems, which inspired this implementation.

References

1. España-Boquera, S., Castro-Bleda, M. J., Gorbe-Moya, J., & Zamora-Martínez, F. (2011). Improving offline handwritten text recognition with hybrid HMM/ANN models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(4), 767–779.
2. Marti, U. V., & Bunke, H. (2002). The IAM-database: An English sentence database for offline handwriting recognition. *International Journal on Document Analysis and Recognition*, 5(1), 39–46.
3. Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2), 257–286.
4. Bourlard, H. A., & Morgan, N. (1994). *Connectionist speech recognition: A hybrid approach*. Springer Science & Business Media.
5. Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *Proceedings of ICML*, 369–376.
6. Graves, A., & Schmidhuber, J. (2009). Offline handwriting recognition with multi-dimensional recurrent neural networks. *Advances in Neural Information Processing Systems*, 21, 545–552.
7. Shi, B., Bai, X., & Yao, C. (2017). An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11), 2298–2304.
8. Bluche, T., Ney, H., & Kermorvant, C. (2014). A comparison of sequence-trained deep neural networks and recurrent neural networks optical modeling for handwriting recognition. *International Conference on Statistical Language and Speech Processing*, 199–210.

A Appendix: Code Snippets

A.1 Debug Training Script

The following code excerpt shows the single-sample overfitting test used to verify model capacity:

```
1 def main():
2     print(f"--- DEBUG MODE: Overfitting One Image ---")
3     print(f"Using Device: {DEVICE}")
4
5     full_dataset = IAMDataset(FEATURE_DIR, XML_DIR)
6     debug_subset = Subset(full_dataset, [0])
7     train_loader = DataLoader(debug_subset, batch_size=1, shuffle=False)
8
9     _, _, sample_text = full_dataset[0]
10    print(f"Target Text to Learn: '{sample_text}'")
11
```

```

12 model = ANN(num_classes=TOTAL_STATES).to(DEVICE)
13 hmm = HybridHMM(num_classes=TOTAL_STATES)
14 optimizer = optim.Adam(model.parameters(), lr=LR)
15 criterion = nn.NLLLoss()
16
17 model.train()
18
19 for epoch in range(1, EPOCHS + 1):
20     for features, targets, text in train_loader:
21         features = features.to(DEVICE).squeeze(0)
22         targets = targets.to(DEVICE).squeeze(0)
23
24         optimizer.zero_grad()
25         outputs = model(features)
26         loss = criterion(outputs, targets)
27         loss.backward()
28
29         # Monitor gradient magnitude
30         total_grad = sum(p.grad.abs().mean().item()
31                         for p in model.parameters() if p.grad is not
32                         None)
33
34         print(f"Avg Gradient Magnitude: {total_grad:.6f}")
35
36         torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm
37         =5.0)
38
39         optimizer.step()
40
41         if epoch % 10 == 0:
42             with torch.no_grad():
43                 hmm_str = hmm.decode(outputs.cpu().numpy())
44             print(f"Epoch {epoch} | Loss: {loss.item():.4f}")
45             print(f"Prediction: '{hmm_str}'")

```

Listing 5: debug_main.py - Core Training Loop

A.2 Dynamic State Allocation

```

1 STATE_COUNTS = {
2     ' ': 1, # Single state for space (silence)
3     '.,': 2, ',': 2, '"': 2, '-': 2, ':': 2, ';': 2,
4     'i': 2, 'I': 2, 'l': 2, 'j': 2, '1': 2, '!': 2,
5     'm': 5, 'w': 5, 'M': 5, 'W': 5,
6     'f': 4, 't': 3, 'r': 3,
7 }
8 DEFAULT_STATES = 3
9
10 # Build cumulative state mapping
11 CHAR_TO_STATE_RANGES = {}
12 TOTAL_STATES = 0
13 for char in CHARS:
14     count = STATE_COUNTS.get(char, DEFAULT_STATES)
15     CHAR_TO_STATE_RANGES[char] = (TOTAL_STATES, count)
16     TOTAL_STATES += count

```

Listing 6: Character-Specific State Counts